

计算机图形学课程实验报告

基于 OpenGL 的三维图形引擎 与 Minecraft 风格游戏

姓 名： 杨征雨

学 号： 3230105712

学 院： 竺可桢学院

专 业： 人工智能

2026 年 6 月 29 日

目录

1 项目概述	3
1.1 技术栈	3
1.2 工程结构	3
2 基本功能实现	3
2.1 基本体素建模	3
2.2 三维网格导入导出 (OBJ)	4
2.3 材质与纹理	4
2.3.1 纹理图集	4
2.4 几何变换	4
2.5 光照模型	4
2.5.1 光源编辑	5
2.6 场景漫游	5
2.7 动画播放	5
2.7.1 屏幕截图	5
3 扩展功能实现	6
3.1 NURBS 曲面建模	6
3.2 实时碰撞检测	6
3.2.1 核心算法 (src/collision/aabb.h)	6
3.2.2 门碰撞系统	6
3.2.3 其他碰撞应用	6
3.3 实时阴影映射	7
3.3.1 Shadow Pass (renderShadowPass())	7
3.3.2 PCF 软阴影 (Fragment Shader 中)	7
3.4 完整三维游戏	7
3.4.1 核心玩法	7
3.4.2 视觉特性	7
3.5 对象表达能力 (门、窗、墙)	8
4 核心算法详解	8
4.1 地形生成算法	8
4.2 碰撞检测算法	8
4.3 门扇旋转 AABB 碰撞	9
5 运行与编译	9
5.1 编译方法	9
5.2 操作说明	9

6 总结与展望

10

1 项目概述

本项目实现了一个基于 OpenGL 3.3 和 C++17 的三维图形渲染引擎，并在此基础上构建了一款具有可玩性的 Minecraft 风格第一/第三人称沙盒游戏 `final_project`。

1.1 技术栈

- 图形 API: OpenGL 3.3 Core Profile
- 语言: C++17
- 窗口管理: GLFW 3.x
- 数学库: GLM (OpenGL Mathematics)
- GUI: Dear ImGui
- 图像加载: stb_image
- 构建系统: CMake 3.15+

1.2 工程结构

项目采用模块化设计，核心代码位于 `src/` 目录下：

- `src/base/` —— 基础组件（应用框架、相机、纹理、FBO、天空盒等）
- `src/mesh/` —— 网格数据结构
- `src/primitives/` —— 基本体素生成（立方体、球、圆柱等）
- `src/obj_loader/` —— OBJ 文件导入导出
- `src/render_util/` —— Phong 光照着色器封装
- `src/collision/` —— AABB 碰撞检测
- `src/animation/` —— 骨骼动画系统
- `src/nurbs/` —— NURBS 曲面建模
- `src/camera/` —— 自由相机
- `src/orbit_control/` —— 轨道控制器
- `src/game/` —— 游戏逻辑（世界生成、碰撞、敌人、箭塔等）

2 基本功能实现

2.1 基本体素建模

引擎支持以下基本图元的程序化生成（`src/primitives/primitives.cpp`）：

- 立方体 (Cube) —— 带法线的 36 顶点立方体，支持任意尺寸缩放
- 球体 (Sphere) —— 基于经纬度细分的三角网格
- 圆柱体 (Cylinder) —— 可指定上下半径和段数
- 圆锥体 (Cone) —— 圆柱体的退化情况（一端半径为 0）
- 多面棱柱 / 棱台 —— 通过正多边形底面扫掠实现

游戏场景中大量使用立方体实例化渲染 (instanced rendering), 单次 draw call 即可绘制同一类型的所有方块。门框由 4 个缩放立方体 (左右柱 + 上下梁) 拼接而成。

2.2 三维网格导入导出 (OBJ)

src/obj_loader/obj_loader.cpp 实现了完整的 Wavefront OBJ 格式读写:

- 导入: 解析 v, vt, vn, f 行, 支持三角面和四边面
- 导出: Save() 函数将内存中的 Element 结构序列化为标准 OBJ 文件
- UI 集成: 游戏内可从 media/obj/ 目录浏览和选择 OBJ 模型放置在场景中
- 场景导出: 可将当前所有方块体素一键导出为 exported_scene.obj

2.3 材质与纹理

引擎支持两种着色器模式:

- 纯色模式 (SimpleLitShader): 每种方块类型有独立的 Phong 材质颜色
- 纹理模式 (TexturedLitShader): 使用程序化生成的纹理图集 (atlas)

2.3.1 纹理图集

系统在运行时生成 1024×1024 像素的纹理图集 (buildTextureAtlas()), 分为 8 列 (每 tile 128×128), 包含:

- 草方块 (顶面绿 + 侧面土 + 底面土, 共 3 tile)
- 泥土、石头、木头、树叶、沙子——各占 1 tile
- 水面——带半透明的蓝色 tile
- 总计 9 种方块类型的完整纹理

Fragment Shader 中根据顶点法线方向动态选择对应的 tile (顶面 / 底面 / 侧面), 实现类似原版 Minecraft 的方块纹理映射。

2.4 几何变换

系统通过 GLM 库支持完整的仿射变换管线:

- TRS 模型矩阵: 每个物体的 `model = translate * rotate * scale`
- 用户交互: ImGui 面板可实时编辑选中物体的平移、旋转、缩放参数
- 相机变换: 透视投影 (FOV 60°) + 观察矩阵, 支持第一/第三人称切换
- 光源变换: 太阳方向、点光源位置均可通过 UI 实时调整

2.5 光照模型

基于 Blinn-Phong 模型实现, 包含三层光源 (完整 shader 见 src/render_util/simple_lit.h), 公式如下:

$$\begin{aligned}
 L = & \underbrace{k_a \cdot C \cdot (0.5 + 0.5N_y)}_{\text{半球环境光}} \\
 & + S \cdot \underbrace{[k_d \cdot C \cdot (N \cdot L) + k_s \cdot (R \cdot V)^\alpha]}_{\text{平行光 diffuse+specular}} \\
 & + \underbrace{\frac{I}{k_c + k_l d + k_q d^2} \cdot C_p \cdot [\text{diffuse} + \text{specular}]}_{\text{点光源}}
 \end{aligned}$$

其中 S 为阴影因子（见第 3.3 节）。最终输出经过 Gamma 校正 ($\gamma = 2.2$)。

2.5.1 光源编辑

通过 ImGui 面板可实时编辑：

- 太阳方向向量、颜色（暖色调 1.0/0.95/0.85）
- 环境光强度
- 点光源位置、颜色、强度
- 点光源衰减系数 (k_c, k_l, k_q)
- 镜面高光指数、颜色
- 阴影开关及阴影偏移量

2.6 场景漫游

提供三种相机模式：

- **第三人称轨道相机（默认）**：鼠标右键拖拽旋转（Orbit），滚轮缩放（Zoom In/Out），中键平移（Pan）。基于 Arcball 旋转实现，操作流畅
- **第一人称**：WASD 移动 + 鼠标控制视角，带碰撞检测，适合游戏体验。按 F 键切换
- **全局缩放（Zoom To Fit）**：场景加载时自动调整相机距离以适应场景包围盒

相机参数（FOV、近远裁剪面）均可在代码中调整。支持 `glm::perspective` 和 `glm::lookAt` 标准投影。

2.7 动画播放

引擎内置完整的骨骼动画系统（`src/animation/`）：

- **骨骼层次结构**：Skeleton 类维护关节树，支持递归的世界矩阵计算
- **蒙皮网格**：SkinnedMesh 支持最多 4 骨骼混合的顶点蒙皮（vertex skinning），双四元数插值
- **动画片段**：AnimationClip 导入 GLTF 动画数据，Animator 控制播放、混合、过渡
- **人形角色**：HumanoidBuilder 程序化生成人形骨骼 + mesh，Humanoid 类管理动画状态机
- **游戏集成**：主角在场景中播放 idle/walk/jump 动画，状态由 `_animState` 控制

2.7.1 屏幕截图

按 P 键可保存当前帧到 `screenshots/` 目录，使用 `stb_image_write` 输出 PNG 格式。

3 扩展功能实现

3.1 NURBS 曲面建模

src/nurbs/nurbs.h 中实现了完整的 NURBS 曲面自主算法:

- **B-spline 基函数**: 实现了 Cox-de Boor 递推公式
- **NURBS 曲面求值**: 双参数域采样, 支持任意阶数 (degree) 和控制点网格
- **可视化**: 可切换线框/实体模式, 动态调整曲面细分精度
- **独立演示**: 08_nurbs 示例程序展示 NURBS 曲面建模效果

3.2 实时碰撞检测

碰撞检测基于空间几何 AABB (Axis-Aligned Bounding Box), 具有完整的实用性:

3.2.1 核心算法 (src/collision/aabb.h)

- **AABB 定义**: `AABB::fromMinMax(min, max)` 构造轴对齐包围盒
- **重叠判断**: 六轴分离检测 (SAT 在轴对齐情况下的简化)
- **滑动碰撞**: `collideAndSlide()` 实现经典的 AABB vs 网格滑动碰撞解析
 - 先 Y 轴、再 X/Z 轴分步检测, 模拟沿表面滑动的效果
 - 玩家半宽收窄处理 (margin) 避免卡墙角
 - 自动检测 `onGround` 状态用于跳跃逻辑

3.2.2 门碰撞系统

专门实现了门的碰撞状态机 (`resolvePlayerMovement()` 中):

- **关闭状态**: 门区域 (含门扇 + 门框) 作为一个完整实心 AABB, 不可通行
- **打开/动画中**:
 - 门扇碰撞使用旋转后 AABB ($\cos \theta, \sin \theta$ 计算真实半轴), 按最短重叠轴推出
 - 左右门柱 AABB (`lb, rb`) 始终阻挡玩家
 - 顶部门梁 AABB (`tb`) 阻止从上方穿过
 - 底部门槛不阻挡, 允许玩家自由出入
- **动画过渡**: 门从 0° 平滑旋转到 90° (或反向), 动画过程中碰撞 AABB 连续更新
- **交互**: 按 0 键在 3.5 范围内开关最近的门

3.2.3 其他碰撞应用

- 玩家移动、跳跃、重力均经过碰撞检测约束
- 投射物 (箭矢) 与方块碰撞检测
- 敌人寻路避开障碍物

3.3 实时阴影映射

实现了标准的 Shadow Mapping + PCF 软阴影：

3.3.1 Shadow Pass (`renderShadowPass()`)

- 2048×2048 深度纹理 (`GL_DEPTH_COMPONENT`, float)
- 正交投影光源空间矩阵：以玩家位置为中心，范围 30×30
- 使用 `glCullFace(GL_FRONT)` 消除 peter-panning 伪影
- 深度着色器仅写入深度，不输出颜色

3.3.2 PCF 软阴影 (Fragment Shader 中)

- $5 \times 5 = 25$ 次邻域采样，加权平均得到柔和阴影边缘
- 手动深度比较 (非硬件 PCF)，完整的 bias 控制
- 阴影仅作用于太阳平行光 (点光源无阴影，避免性能问题)

3.4 完整三维游戏

基于引擎构建了可玩的 Minecraft/TD 混合游戏 `final_project`：

3.4.1 核心玩法

- **世界生成**：程序化噪声地形 (multi-octave 噪声)，32×32 区块，包含树木自动放置
- **体素编辑**：右键放置方块，左键破坏 (掉落物品可拾取)
- **塔防机制**：
 - 放置箭塔防御从四面八方涌来的敌人
 - 敌人 (灰骑士模型) 沿地形寻路向玩家移动
 - 波次管理系统 (`WaveManager`)，难度递增
 - HUD 显示波次、生命值、得分
- **第一/第三人称切换**：按 F 键无缝切换
- **背包系统**：按 E 键打开背包，九宫格热键栏
- **场景装饰**：悬浮地球仪 + 轨道旋转的基本图元展示

3.4.2 视觉特性

- 程序化生成的天空盒 (雪山 + 湖景)，6 面无缝拼接
- 实时阴影覆盖所有方块和角色
- 半透明水面渲染 (alpha blending)
- 昼夜循环效果 (通过调整太阳方向实现)
- 角色骨骼动画 (idle/walk/jump 平滑过渡)

3.5 对象表达能力（门、窗、墙）

实现了具有实际功能差异的建筑对象系统：

- 门：
 - 由门框（左右柱 + 上梁 + 下梁）和门扇组成
 - 关闭时：实心碰撞体积，不可通行
 - 打开时：门扇旋转 90°，碰撞 AABB 实时更新；左右柱和顶梁保持碰撞
 - 按键交互开关（O 键）
 - 木质配色（深棕色门框 + 浅棕色门扇）
- 窗：
 - 半透明玻璃方块（BT_WATER 类型复用，带 alpha）
 - 可被破坏（掉落物品）
 - 无碰撞体积（玩家可穿过）
- 墙：
 - 石头/泥土方块堆叠，始终不可通行
 - 标准 AABB 碰撞检测

这三种建筑对象的行为差异明确：**门**可交互开关、**窗**可穿过可破坏、**墙**始终阻塞。符合“关闭的门不可通行，可以按键打开，打开后可通行；窗可打碎、翻越；墙始终不可通行”的要求。

4 核心算法详解

4.1 地形生成算法

采用多层叠加的 2D 噪声函数（伪随机哈希 + 平滑插值）：

```
1 float MinecraftGame::getTerrainHeight(float wx, float wz) const {  
2     float h = 0;  
3     h += noise2D(wx * 0.02f, wz * 0.02f) * 12.0f;    // 大尺度起伏  
4     h += noise2D(wx * 0.05f, wz * 0.05f) * 5.0f;    // 中等尺度  
5     h += noise2D(wx * 0.12f, wz * 0.12f) * 2.0f;    // 细节  
6     return h;  
7 }
```

根据高度值分层放置方块：低于高度 → 石头，等于高度 -1 → 泥土，等于高度 → 草方块。低于水位线 → 沙地/水。

4.2 碰撞检测算法

核心采用分离轴定理（SAT）的简化版——AABB vs AABB 六轴检测：

$$\text{overlap}(A, B) \iff \begin{cases} A.\min_x < B.\max_x \wedge A.\max_x > B.\min_x \\ A.\min_y < B.\max_y \wedge A.\max_y > B.\min_y \\ A.\min_z < B.\max_z \wedge A.\max_z > B.\min_z \end{cases}$$

滑动碰撞分三步 ($Y \rightarrow X \rightarrow Z$ 顺序), 每一步独立检测碰撞并允许平行于墙面的滑动, 实现自然的贴墙行走效果。

4.3 门扇旋转 AABB 碰撞

门扇绕合页轴旋转角度 θ , 其 AABB 半轴为:

$$\begin{aligned} h_x &= |\cos \theta| \cdot \frac{W}{2} + |\sin \theta| \cdot \frac{T}{2} \\ h_z &= |\sin \theta| \cdot \frac{W}{2} + |\cos \theta| \cdot \frac{T}{2} \end{aligned}$$

其中 W 为门扇宽度 (1.76), T 为厚度 (0.21)。resolvePlayerMovement() 中用 $\cos \theta$ 和 $\sin \theta$ 计算真实 AABB, 沿最短重叠轴推出玩家, 保证任意旋转角度下碰撞均精确有效。

5 运行与编译

5.1 编译方法

```
1 cd final
2 mkdir build && cd build
3 cmake .. -G "Visual_Studio_17_2022"
4 cmake --build . --config Release
```

编译产物位于 build/bin/Release/:

- final_project.exe —— 主游戏程序
- 01_primitives.exe ~ 09_skeletal.exe —— 各模块演示程序

5.2 操作说明

- **WASD**: 移动 **Space**: 跳跃
- **鼠标左键**: 破坏方块 **鼠标右键**: 放置方块
- **F**: 切换第一/第三人称 **E**: 打开背包
- **O**: 开关最近的门 **P**: 截图
- **1-9**: 快捷栏选择方块类型
- **滚轮**: 第三人称缩放 **右键拖拽**: 旋转视角
- **Esc**: 释放鼠标 **Settings 面板**: 调整光源参数

6 总结与展望

本项目实现了一个功能完整的三维图形引擎，覆盖了课程要求的全部基本功能（体素建模、OBJ 导入导出、材质纹理、几何变换、Phong 光照、场景漫游、动画截图）和多项扩展功能（NURBS 曲面、AABB 碰撞检测、PCF 阴影映射、完整游戏、门/窗/墙对象系统）。

引擎采用模块化设计，各子系统（渲染、碰撞、动画、游戏逻辑）解耦清晰，便于扩展和维护。通过合理的算法选择（如 instanced rendering 优化方块绘制、PCF 软阴影提升画质、旋转 AABB 处理可动碰撞体）在功能和性能之间取得了良好平衡。

未来可扩展方向：全局光照（Voxel GI）、物理模拟（Bullet 集成）、网络多人联机、更丰富的生物群系等。